DOCUMENTO TÉCNICO OFICIAL · VERSIÓN 1.0

ATLAS GYM
Manual Técnico y Guía Maestra de Estudio

Arquitectura de Software, Código de Producción y Fundamentos de Ingeniería: De Cero a Producción en una API REST y Frontend Moderno

Equipo de Desarrollo: Lucas · Bruno · Augusto
Proyecto: Plataforma ATLAS Gym (Web & API REST)
Ecosistema Front: Next.js · TypeScript · Tailwind CSS
Ecosistema Back: Spring Boot 3.4.3 · Java 17 · MySQL 8
Seguridad: JWT (HMAC-SHA256) · BCrypt · Stateless
Fecha: Octubre 2026

Tabla de Contenidos

1. Prólogo y Filosofía del Proyecto

2. Visión General de la Arquitectura (MVC Desacoplado)

3. Fundamentos Cruciales de Estudio: Desde Cero Absoluto

3.1. ¿Cómo funciona la Web? El Protocolo HTTP, Verbos y Códigos de Estado

3.2. Formato JSON: La lengua franca entre Frontend y Backend

3.3. Stateless (Sin Estado): Por qué no guardamos sesiones en memoria

3.4. CORS: El mecanismo de seguridad del navegador

3.5. Variables de Entorno y Portabilidad en Equipos

4. El Ecosistema Frontend: Next.js, React y TypeScript

4.1. Por qué Next.js frente a React simple

4.2. TypeScript y la garantía de tipos en compilación

4.3. El patrón de Estado Centralizado (lib/store.tsx)

4.4. Testing automatizado con Vitest (114 tests)

5. El Ecosistema Backend: Spring Boot, Java 17 y MySQL

5.1. La Arquitectura en Capas Estricta (Controlador, Servicio, Repositorio, Modelo)

5.2. Inyección de Dependencias y Principio IoC

5.3. JPA y Hibernate: Mapeo Objeto-Relacional (ORM) y Relaciones

5.4. DTOs con Records de Java 17: Por qué NUNCA exponer Entidades

5.5. @Transactional: Integridad ACID y Rollback Automático

5.6. Manejo Global de Errores con @RestControllerAdvice

6. Seguridad y Criptografía Moderna

6.1. Hashing de Contraseñas con BCrypt (Salt y funciones unidireccionales)

6.2. JSON Web Tokens (JWT): Anatomía, firma y expiración

6.3. Cadena de Filtros de Spring Security (OncePerRequestFilter)

6.4. Control de Acceso Basado en Roles (RBAC con @PreAuthorize)

7. Las 7 Reglas de Negocio Críticas de ATLAS Gym

7.1. Anti-fraude del Tótem (ventana deslizante de 100 minutos)

7.2. Snapshot Inmutable de Pagos

7.3. Baja Administrativa Lógica (@PreRemove)

7.4. Estado de Cuenta Dinámico (AL_DIA, PENDIENTE, MOROSO, INACTIVO)

7.5. Protección de Planes con Alumnos Activos

7.6. Propagación de Rutinas por Clave Foránea

7.7. Workout Tracker y Sobrecarga Progresiva

8. Mapa Completo de Endpoints REST del Backend

9. Guía de Conexión Frontend & Backend (Next.js & Spring Boot)

10. Plan de Estudio y Preguntas de Entrevista para el Equipo

1. Prólogo y Filosofía del Proyecto

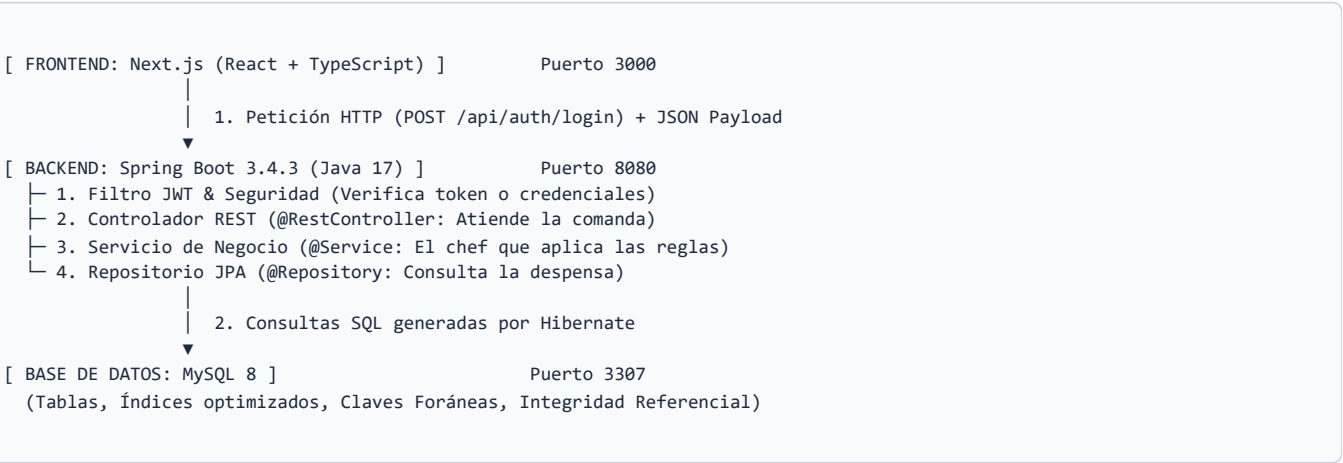
Este proyecto representa un hito fundamental para **Lucas, Bruno y Augusto**: es su **primer trabajo profesional remunerado** y, al mismo tiempo, el laboratorio definitivo para consolidar conocimientos de ingeniería de software full-stack.

En el desarrollo de software profesional existe un estándar ético inquebrantable: *un ingeniero jamás entrega un sistema sin comprender con total profundidad cómo funciona cada una de sus partes*. No basta con que una pantalla se vea bonita o que un botón responda; es indispensable saber por qué se eligió cada patrón de diseño, qué ocurre en la base de datos cuando se pulsa un botón y cómo proteger la información sensible del cliente.

> **El Objetivo de este Documento**
> Servir como manual de referencia técnica del proyecto y como **guía de estudio maestro** para que todo el equipo domine desde los conceptos más elementales hasta las arquitecturas más complejas, preparándolos tanto para defender este trabajo como para superar cualquier entrevista técnica futura.

2. Visión General de la Arquitectura

El sistema ATLAS Gym se diseñó bajo una arquitectura **MVC Desacoplada (API REST)**, dividiendo el sistema en dos proyectos completamente independientes que dialogan mediante HTTP y JSON:



¿Por qué esta separación y no un sistema todo-en-uno?

- **Independencia de despliegue:** el equipo de frontend puede actualizar estilos, textos y componentes sin necesidad de recompilar ni reiniciar el servidor de Spring Boot.
- **Múltiples clientes:** el backend no le pertenece solo a una página web. Si el día de mañana el gimnasio instala un Tótem físico en una Raspberry Pi o lanza una aplicación móvil en Flutter o React Native, *utilizan exactamente los mismos endpoints sin tocar una sola línea de Java*.
- **Escalabilidad y Seguridad:** la base de datos MySQL nunca queda expuesta a internet; solo el backend tiene acceso a ella.

3. Fundamentos Cruciales de Estudio: Desde Cero Absoluto

3.1. ¿Cómo funciona la Web? El Protocolo HTTP

Cada vez que el navegador interactúa con el servidor, intercambian mensajes bajo el protocolo **HTTP (Hypertext Transfer Protocol)**. Este modelo es estrictamente **Petición-Respuesta (Request-Response)**: el cliente pide algo, el servidor procesa y responde.

Los Verbos HTTP (Intención de la Petición):

Verbo	Significado	¿Modifica datos?	Ejemplo en ATLAS Gym
`GET`	Consultar / Leer datos	No (Es idempotente)	`GET /api/alumnos` (Lista los socios)
`POST`	Crear un nuevo recurso	Sí (Inserta en DB)	`POST /api/alumnos` (Crea un nuevo socio)
`PUT`	Actualizar un recurso existente	Sí (Modifica fila)	`PUT /api/alumnos/3` (Edita el socio con ID 3)
`DELETE`	Eliminar o dar de baja	Sí (Baja lógica)	`DELETE /api/alumnos/3` (Baja lógica del socio)

Códigos de Estado HTTP (Status Codes):

- `200 OK`: Operación exitosa con datos devueltos.
- `201 Created`: Nuevo registro creado satisfactoriamente en la base de datos.
- `204 No Content`: Operación exitosa que no necesita devolver datos (ideal para borrados o bajas).
- `400 Bad Request`: La petición tiene errores de sintaxis o faltan campos obligatorios.
- `401 Unauthorized`: Falta autenticación (no enviaste token o las credenciales no son válidas).
- `403 Forbidden`: El usuario está autenticado, pero su rol no tiene permiso para ese recurso.
- `404 Not Found`: El recurso solicitado no existe en la base de datos.
- `409 Conflict`: Conflicto con el estado de la base de datos (ej. DNI ya registrado).
- `500 Internal Server Error`: Error no controlado en el servidor (un bug que debe corregirse).

3.2. Formato JSON: La lengua franca

JSON (JavaScript Object Notation) es el formato estándar mundial para transferir datos porque es texto plano, ligero y entendible tanto por humanos como por computadoras. Java serializa sus objetos en JSON y Next.js los deserializa en objetos JavaScript de manera transparente.

3.3. Stateless (Arquitectura Sin Estado)

En arquitecturas antiguas, el servidor guardaba en su memoria RAM una "sesión" para recordar quién era cada usuario. En arquitecturas modernas **Stateless**, el servidor **no guarda nada en memoria**. Cada petición incluye un token criptográfico (JWT) en la cabecera. Si el servidor se reinicia o se crean 5 réplicas del servidor, el usuario ni se entera porque la validez de su sesión viaja consigo mismo en el token.

3.4. CORS (Cross-Origin Resource Sharing)

CORS es un mecanismo de protección de **los navegadores**. Si tu frontend corre en `http://localhost:3000` y tu backend en `http://localhost:8080`, el navegador detecta que tienen diferentes puertos (orígenes distintos). Por seguridad, bloquea la petición a menos que el backend envíe cabeceras HTTP explícitas autorizando a `localhost:3000`. Esto quedó configurado en `SecurityConfig.java`.

3.5. Variables de Entorno y Portabilidad

En software profesional nunca se escriben contraseñas o puertos fijos en el código. En `application.properties` configuramos `${DB_PORT:3307}`. Si una variable de entorno `DB_PORT` existe en el sistema operativo, Spring la toma; de lo contrario, usa el puerto 3307 por defecto. Esto permite que el proyecto se ejecute en la máquina de Lucas, de Bruno o en un servidor en la nube sin tocar una sola línea de código.

4. El Ecosistema Frontend: Next.js, React y TypeScript

4.1. Por qué Next.js frente a React simple

Un proyecto con React puro descarga un archivo HTML vacío y un JavaScript enorme, dejando todo el renderizado al navegador. **Next.js** es un framework empresarial sobre React que ofrece:

- **Enrutamiento basado en archivos:** crear una carpeta `app/alumnos/page.tsx` crea automáticamente la ruta web `/alumnos` sin necesidad de configurar librerías de rutas externas.
- **Optimización de medios:** el componente `<Image />` redimensiona, convierte a formatos modernos (WebP) y retrasa la carga de imágenes que están fuera de la pantalla.
- **División entre servidor y cliente:** componentes del servidor rápidos y componentes del cliente interactivos mediante la directiva `"use client"`.

4.2. TypeScript y la Garantía de Tipos

TypeScript agrega un sistema de tipado estricto sobre JavaScript. Si el backend define que un socio tiene `dni: string` y en el frontend intentás acceder a `socio.cedula`, el compilador arroja un error antes de que el código llegue al navegador. En `lib/types.ts` están modelados todos los contratos de datos del sistema.

4.3. El Patrón de Estado Centralizado (`lib/store.tsx`)

Bruno estructuró el frontend con un patrón sumamente profesional: **Context + Custom Hook (`useAppData`)**.

> **La Sabiduría Arquitectónica del Store**
>
> Las pantallas individuales (como `/alumnos`, `/finanzas`, `/asistencias`) **no hacen peticiones HTTP directamente**. Le piden y envían datos exclusivamente al Store. Esto significa que para conectar el backend con el frontend, **no hay que reescribir las páginas visuales**; solo cambiamos las funciones internas del Store para que hagan `fetch()` a Spring Boot.

4.4. Testing Automatizado con Vitest

El frontend cuenta con **114 tests automatizados** que verifican reglas lógicas clave: cálculo de cuotas vencidas, consistencia de planes y validaciones de formularios. Esto asegura que cualquier refactorización o cambio de código no rompa funcionalidades previas.

5. El Ecosistema Backend: Spring Boot, Java 17 y MySQL

5.1. La Arquitectura en Capas Estricta

En Spring Boot el código se organiza en capas según su responsabilidad única:

Capa	Paquete	Anotación	Responsabilidad
Controlador	`com.atlas.backend.Controlador`	`@RestController`	Punto de entrada HTTP. Valida la petición entrante, llama al servicio y devuelve el código de estado adecuado (200, 201, 204).
Servicio	`com.atlas.backend.Servicio`	`@Service`	**El cerebro del negocio.** Aplica las reglas del gimnasio, valida duplicados, maneja transacciones con `@Transactional`.
Repositorio	`com.atlas.backend.Repositorio`	`@Repository`	Acceso a la base de datos. Extiende `JpaRepository` y ejecuta consultas SQL/JPQL optimizadas.
Modelo	`com.atlas.backend.Modelo`	`@Entity`	Representación exacta de las tablas de MySQL mediante clases Java y anotaciones de JPA.
DTO	`com.atlas.backend.Dto`	`record`	Objetos inmutables de transferencia de datos. Son los contratos que viajan por la red hacia y desde Next.js.
Excepciones	`com.atlas.backend.Excepciones`	`@RestControllerAdvice`	Captura errores de negocio y validación en toda la app y devuelve respuestas JSON limpias y predecibles.

5.2. Inyección de Dependencias (IoC)

En lugar de que cada clase cree a sus colaboradoras con `new`, Spring Boot utiliza el principio de **Inversión de Control (IoC)**. Cuando el servidor arranca, Spring crea una única instancia de cada clase anotada (los *Beans*) y las inyecta a través del constructor. Esto garantiza bajo acoplamiento y alta facilidad para hacer pruebas unitarias.

5.3. JPA y Hibernate: El Mapeo Objeto-Relacional

Hibernate traduce automáticamente nuestros objetos Java a tablas y columnas de MySQL:

- `@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)`: Una Rutina tiene muchos Días. Si guardamos la Rutina, Hibernate guarda todos sus días hijos automáticamente. Si borramos un día de la lista en Java, Hibernate borra la fila huérfana en la base de datos.
- `@ManyToOne(fetch = FetchType.LAZY)`: Muchos Alumnos tienen un mismo Plan. Con carga perezosa (*Lazy*), Hibernate no carga los datos del plan a menos que se los pidamos explícitamente, ahorrando memoria y tiempo de CPU.

5.4. DTOs: Por qué NUNCA exponer Entidades

> **Regla de Oro de Seguridad y Rendimiento**
>
> Jamás se debe devolver una entidad `@Entity` directamente en un controlador REST. Si devolvés `Alumno`, Jackson intentará serializar sus `Pagos`, los pagos serializarán al alumno, y se generará un bucle infinito que tumba el servidor con `StackOverflowError`. Además, la entidad `Usuario` contiene el hash de la contraseña, el cual jamás debe viajar al navegador. Los **DTOs (Data Transfer Objects)** resuelven esto enviando únicamente los datos necesarios y limpios.

5.5. `@Transactional`: Integridad ACID

Anotar un método con `@Transactional` asegura que todas las operaciones dentro de él se ejecuten como una única unidad atómica. Si se guardan 3 series de un ejercicio pero la cuarta falla por un error de conexión, **Spring Boot ejecuta un Rollback automático** y deja la base de datos intacta.

6. Seguridad y Autenticación Moderna

6.1. Hashing de Contraseñas con BCrypt

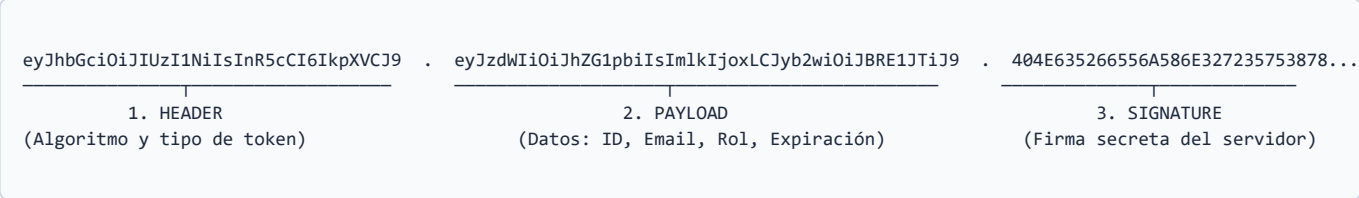
Las contraseñas de los socios y administradores **nunca se guardan en texto plano**. Si alguien accede a la base de datos, solo verá una cadena ilegible como:

```
$2a$10$e8w9u6V0lqGZ9zJ4X9H8O.k2qM6j5P2B1F4Y7U8I9O0P1Q2R3S4T5
```

BCrypt es una función de hash **unidireccional**: nadie puede "desencriptarla". Para validar el login, el servidor toma la contraseña que escribió el usuario, le aplica el mismo algoritmo de hash con el Salt correspondiente, y verifica si los resultados coinciden.

6.2. JSON Web Tokens (JWT) en Detalle

Al hacer login exitoso, el servidor emite un **Token JWT** firmado criptográficamente con el algoritmo **HMAC-SHA256**. El token está compuesto por tres secciones:



Si un atacante en el navegador altera el Payload para cambiar su rol de `ALUMNO` a `ADMIN`, la firma secreta deja de coincidir inmediatamente y el servidor rechaza la petición con código **403 Forbidden**.

6.3. Cadena de Filtros de Spring Security

Cada solicitud que llega a Spring Boot pasa por `JwtAuthenticationFilter` antes de llegar a cualquier controlador. El filtro extrae el token del encabezado `Authorization: Bearer <token>`, valida la firma, comprueba que no haya expirado y establece los permisos en el contexto de seguridad de Spring.

7. Las 7 Reglas de Negocio Críticas de ATLAS Gym

Regla	Servicio Responsable	Problema que Resuelve y Funcionamiento Técnico
1. Anti-fraude del Tótem (100 min)	`AsistenciaService`	Evita que un socio digite su DNI y se lo pase a un amigo que viene detrás. Si ya tiene un ingreso registrado en el día con menos de 100 minutos de diferencia, se rechaza con `"Tu sesión continúa activa"`. Si pasaron 100 min o más, se permite un reingreso legítimo.
2. Snapshot Inmutable de Pagos	`PagoService`	Si el gimnasio sube el precio de la cuota en noviembre, los recibos de octubre no deben cambiar de precio. Al cobrar, se guarda una copia estática en texto de `alumnoNombreHistorico` y el `monto` cobrado en ese momento.
3. Baja Administrativa Lógica	`AlumnoService`	Nunca se borra físicamente una fila con `DELETE` para no perder el historial de pagos y entrenamientos. Se ejecuta `darDeBaja()` (`activo = false`). La anotación `@PreRemove` en la entidad arroja una excepción si alguien intenta forzar un borrado físico.
4. Estado de Cuenta Dinámico	`AlumnoService`	No se guarda un valor fijo en la base (para no tener que actualizarlo todos los días 1). Se calcula al vuelo: `AL_DIA` (pagó el mes actual), `PENDIENTE` (período de gracia días 1 al 10), `MOROSO` (día 11 en adelante sin pago) e `INACTIVO` (más de 60 días sin pagar o baja lógica).
5. Protección de Planes Activos	`PlanService`	Si un administrador intenta borrar un plan que todavía tiene alumnos activos asignados, el servicio arroja una `BusinessException` con código **HTTP 409 Conflict** para evitar dejar socios huérfanos.
6. Propagación de Rutinas por FK	`RutinaService`	Los socios apuntan a la rutina por clave foránea. Si un profesor edita los ejercicios de una rutina, el cambio se refleja al instante en el celular de todos los alumnos asignados sin tener que actualizar registro por registro.
7. Workout Tracker y Sobrecarga Progresiva	`SesionEntrenamientoService`	Permite que cuando el alumno inicia su rutina, el endpoint `/api/sesiones/ultima` busque los pesos levantados en su sesión anterior para ese mismo día, facilitando la sobrecarga progresiva en sala.

8. Mapa Completo de Endpoints REST del Backend

Todos los endpoints del backend se encuentran organizados y securizados en el paquete `com.atlas.backend.Controlador`:

Controlador	Método	Ruta	Seguridad	Descripción
AuthController	`POST` `GET`	`/api/auth/login` `/api/auth/perfil`	Público Autenticado	Login con verificación BCrypt y emisión de JWT. Consulta de datos del usuario en sesión.

AlumnoController	`GET`
	`GET`
	`GET`
	`POST`
	`PUT`
	`DELETE`
		`/api/alumnos`
		`/api/alumnos/inactivos`
		`/api/alumnos/{id}`
		`/api/alumnos`
		`/api/alumnos/{id}`
		`/api/alumnos/{id}`
			`ADMIN`
			`ADMIN`
			`ADMIN`
			`ADMIN`
			`ADMIN`
			`ADMIN`
				Lista de socios activos.
				Lista de socios dados de baja.
				Detalle de socio por ID.
				Alta de nuevo socio.
				Modificación de ficha.
				Baja administrativa lógica (HTTP 204).

AsistenciaController	`POST`
	`GET`
	`GET`
	`GET`
		`/api/asistencias/totem`
		`/api/asistencias/aforo`
		`/api/asistencias`
		`/api/asistencias/distribucion`
			Público
			`ADMIN`
			`ADMIN`
			`ADMIN`
				Check-in en el Kiosco por DNI (regla 100 min).
				Contador de socios en sala en tiempo real.
				Historial de ingresos por fecha.
				Picos y distribución horaria (0 a 23 hs).

PagoController	`POST`
	`GET`
	`GET`
		`/api/pagos`
		`/api/pagos`
		`/api/pagos/alumno/{id}`
			`ADMIN`
			`ADMIN`
			`ADMIN`, `ALUMNO`
				Registro de cobro con snapshot inmutable.
				Listado de cuotas (filtro opcional por mes).
				Historial contable de un socio específico.

PlanController	`GET`
	`POST`
	`PUT`
	`DELETE`
		`/api/planes`
		`/api/planes`
		`/api/planes/{id}`
		`/api/planes/{id}`
			Autenticado
			`ADMIN`
			`ADMIN`
			`ADMIN`
				Planes de membresía activos.
				Creación de nuevo plan.
				Actualización de precio o días.
				Eliminación protegida (bloquea si tiene alumnos).

RutinaController	`GET`
	`GET`
	`POST`
	`POST`
		`/api/rutinas/genericas`
		`/api/rutinas/mis-rutinas`
		`/api/rutinas`
		`/api/rutinas/{rId}/asignar/{aId}`
			Autenticado
			`ALUMNO`
			`ADMIN`
			`ADMIN`
				Catálogo de rutinas base del gimnasio.
				Rutina activa del socio autenticado.
				Creación de rutina con días y ejercicios.
				Asignación de rutina a la ficha de un alumno.

SesionController	`POST` `GET`	`/api/sesiones` `/api/sesiones/ultima`	`ADMIN`, `ALUMNO` `ADMIN`, `ALUMNO`	Guardar sesión completada del Workout Tracker. Pesos de la sesión previa para sobrecarga progresiva.

AvisoController	`GET`
	`PUT`
	`POST`
		`/api/avisos/mis-avisos`
		`/api/avisos/{id}/leido`
		`/api/avisos`
			`ALUMNO`
			`ALUMNO`
			`ADMIN`
				Avisos institucionales y privados del alumno.
				Marcar aviso como leído.
				Publicación de comunicado institucional.

VideoTecnicaController	`GET` `GET`	`/api/videoteca` `/api/videoteca/grupo/{grupo}`	Autenticado Autenticado	Catálogo de videos de ejercicios. Filtro por grupo muscular (Pecho, Espalda, etc.).

9. Guía de Conexión Frontend & Backend

Para conectar la interfaz de Next.js con el servidor de Spring Boot se sigue un procedimiento estándar de 3 pasos:

Paso 1: Crear el Cliente HTTP Centralizado (`lib/api.ts`)

```
// lib/api.ts
const API_URL = process.env.NEXT_PUBLIC_API_URL || "http://localhost:8080/api";

export async function apiFetch<T>(endpoint: string, options: RequestInit = {}): Promise<T> {
  const token = typeof window !== "undefined" ? localStorage.getItem("atlas_jwt_token") : null;

  const headers: HeadersInit = {
    "Content-Type": "application/json",
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
    ...options.headers,
  };

  const response = await fetch(`${API_URL}${endpoint}`, { ...options, headers });

  if (!response.ok) {
    const errorData = await response.json().catch(() => ({}));
    throw new Error(errorData.mensaje || `Error HTTP ${response.status}`);
  }

  if (response.status === 204) return null as T;
  return response.json();
}
```

Paso 2: Conectar el Login Real (`app/login/page.tsx`)

Al enviar el formulario de login, se realiza la petición `POST /api/auth/login`. Si las credenciales son válidas, se almacena el token en `localStorage` y se redirige a la pantalla principal.

Paso 3: Conectar el Store Global (`lib/store.tsx`)

En lugar de leer los arrays estáticos de `mock-data.ts`, las funciones de `useAppData()` llaman a `apiFetch("/alumnos")`, `apiFetch("/planes")`, etc., manteniendo intacta la lógica de los componentes de la interfaz.

> **Usuarios Reales Sembrados por el Seeder (DataInitializer)**
>
> - **Administrador:** `admin@atlasgym.com` / Contraseña: `admin123`
> - **Alumno:** `alumno@atlasgym.com` / Contraseña: `alumno123` (asociado a Lucas Pérez, DNI 42123456)

10. Plan de Estudio y Preguntas de Entrevista para el Equipo

A continuación se detalla el temario ordenado por dificultad que **Lucas, Bruno y Augusto** deben dominar para defender este proyecto con solvencia profesional:

Nivel 1: Los Fundamentos Obligatorios

- ¿Qué es una API REST y por qué utilizamos los métodos GET, POST, PUT y DELETE?
- ¿Qué significa que una arquitectura sea Stateless y por qué no guardamos sesiones en memoria?
- ¿Qué es CORS y qué cabeceras envía el navegador en las solicitudes de comprobación previa (*Preflight OPTIONS*)?
- ¿Por qué se utilizan códigos de estado HTTP semánticos (200, 201, 204, 400, 401, 403, 404, 409) en vez de devolver siempre 200 con un mensaje de error?

Nivel 2: Backend y Persistencia

- ¿Cómo funciona la Inyección de Dependencias por constructor y por qué es superior a `@Autowired` sobre atributos?
- ¿Qué es el mapeo relacional en JPA/Hibernate y qué diferencia hay entre `FetchType.LAZY` y `FetchType.EAGER`?
- ¿Qué problema resuelven los DTOs (Records de Java 17) y por qué exponer entidades JPA genera fallos de seguridad y ciclos infinitos en JSON?
- ¿Qué significa el principio ACID y cómo actúa la anotación `@Transactional` ante un error en base de datos?

Nivel 3: Seguridad y Negocio

- ¿Por qué las contraseñas nunca se pueden "desencriptar" y cómo funciona el Salt de BCrypt?
- ¿Cuáles son las tres partes de un JWT y cómo detecta el servidor si el cliente alteró su rol en el Payload?
- ¿Cómo funciona el filtro `OncePerRequestFilter` en la cadena de seguridad de Spring?
- Explicar en detalle las 7 reglas de negocio de ATLAS Gym (Tótem 100 min, snapshot de cobros, baja administrativa lógica, estado de cuenta dinámico).

Nivel 4: Frontend y Arquitectura de Estado

- ¿Por qué se utilizó el patrón Context + Custom Hook en `lib/store.tsx` en lugar de hacer llamadas fetch en cada pantalla?
- ¿Cómo viaja el token JWT en el encabezado `Authorization: Bearer <token>`?
- ¿Cómo se gestionan los mensajes de error devueltos por el `GlobalExceptionHandler` en los formularios de Next.js?

> **Mensaje Final para el Equipo**
>
> Tener un proyecto completo con esta calidad de arquitectura, seguridad por JWT, testing con 114 pruebas y reglas de negocio robustas en su primer trabajo remunerado los ubica **muy por encima del promedio** de desarrolladores junior en el mercado. Estudien este manual, háganse preguntas entre ustedes y siéntanse orgullosos del nivel de ingeniería que construyeron.